

How Computers Work · Section 1

What a Computer Does

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. The Idea of a Computer 1

2. Input, Processing, Output, and Storage 2

3. Hardware and Software 2

4. Everything Inside Is Numbers 3

5. Instructions and Data 4

6. Why Security People Must Understand the Machine 4

7. A Map of This Course 5

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What a computer really is, under the surface.
- The four things every computer does: input, processing, output, and storage.
- The difference between hardware and software.
- Why everything inside a computer is just numbers.
- Why instructions and data are both stored the same way, and why that matters.
- Why a security person has to understand the machine.
- A map of this course.

Before You Start

You need only Security Fundamentals (course 01) and curiosity. No programming, and no prior knowledge of how computers work inside. This course has a little code and a few commands in later lessons, but this first lesson is all ideas.

Why this course exists: in Security Fundamentals you learned *what* to protect and *why*. To actually attack or defend a system, you must understand the machine it runs on. A reverse engineer reads a program's instructions; a forensic analyst recovers data from memory and disks; an exploit developer bends a program's use of memory. All of it rests on the handful of ideas in this course.

---

1. The Idea of a Computer

A computer looks complicated, but the core idea is simple: **a computer is a machine that follows a list of instructions, step by step, very fast, to work with information.**

That is all a computer fundamentally does. It does not think or understand. It follows instructions, one after another, billions of times a second, without ever getting bored or making a different choice than the instructions say. Everything a computer appears to "do", show a video, run a game, send a message, is really a vast number of tiny, simple steps carried out so quickly that the result looks instant and alive.

Two things make a computer powerful:

- **Speed:** a modern processor carries out billions of simple steps every second.
- **Reliability:** it does exactly what the instructions say, every time, the same way.

This is also why computers are a security problem. A computer will follow *whatever* instructions it is given, including an attacker's, if the attacker can get their instructions to run. Much of security is about controlling which instructions a machine will follow, and with whose authority.

2. Input, Processing, Output, and Storage

Every computer, from a phone to a giant server, does four things. Keep these four words; the whole course hangs on them.

Stage	What it means	Everyday example
Input	Information comes in	Typing, tapping, a network message, a sensor
Processing	The computer works on it, following instructions	Calculating, sorting, deciding
Output	A result goes out	The screen, a sound, a network reply
Storage	Information is kept for later	Saving a file, remembering a setting

A simple example: you type a search (input), the computer looks through its data and ranks results (processing), it shows them on the screen (output), and it remembers your recent searches (storage).

Every part of a computer serves one of these four jobs, and so does every security concern. An attacker might poison the **input** (a malicious message), hijack the **processing** (run their own instructions), read the **output** (spy on your screen), or steal the **storage** (copy your files). When you meet a new attack later in the program, it helps to ask: which of the four is it attacking?

3. Hardware and Software

A computer has two sides that work together.

- **Hardware** is the physical machine: the parts you could touch. The processor, the memory chips, the disk, the screen, the keyboard, the network card.
- **Software** is the instructions: the programs that tell the hardware what to do. An operating system, an app, a game, a virus, are all software.

The relationship is simple: **software is a list of instructions, and hardware is the machine that carries them out.** Hardware without software is an inert lump; software without hardware is just a list that does nothing. A program only has an effect when hardware runs it.

Between the two sits a special piece of software called the **operating system** (Windows, macOS, Linux, Android), which manages the hardware and lets other programs use it. Lesson 5 is all about it. For now, picture three layers: the hardware at the bottom, the operating system in the middle managing it, and your apps on top.

This split matters for security because attacks happen at every layer. Some target hardware, most target software, and the most serious often target the operating system in the middle, because whoever controls it controls everything above.

4. Everything Inside Is Numbers

Here is the idea that surprises people most, and that unlocks the rest of the course: **inside a computer, everything is numbers.** Not just sums and prices, but *everything*.

- A letter of text is a number (lesson 3).
- A color in a photo is a set of numbers.
- A sound is a long list of numbers.
- A program's instructions are numbers (lesson 8).
- An image, a video, a document, a web page, all numbers.

A computer has no idea what a number "means". The same number could be a letter, a brightness, or an instruction, depending only on how the computer is told to treat it. This is a deep and powerful fact. It means one kind of machine, working only with numbers, can handle every kind of information in the world.

And the numbers themselves are stored in the simplest possible way: as huge quantities of **on-or-off** switches. Everything, every photo and song and program, is ultimately a pattern of ons and offs. The next lesson shows how that works, with **bits** and **binary**. For now, hold the headline: to a computer, there is no text, no picture, no program, only numbers, stored as switches.

This is why topics like encoding (lesson 3) and file formats (lesson 7) matter so much in security. If everything is numbers, then the *rules* for reading those numbers are everything, and attackers love to break or confuse those rules.

5. Instructions and Data

Since everything is numbers, a computer stores two different kinds of thing in the very same way:

- **Data:** the information a program works on, a photo, a message, a list of prices.
- **Instructions:** the steps the program follows, the program itself.

Both are just numbers in memory. This idea, that a computer holds its instructions and its data together in the same memory, is called the **stored-program** design, and almost every computer ever built works this way. It is why a computer is so flexible: to make it do something new, you do not rebuild the machine, you just give it a new set of instructions (a new program).

But it has a dangerous side, which is the root of a whole class of attacks. If instructions and data live in the same memory and look the same, then a computer can be **tricked into treating data as instructions.** An attacker sends what looks like ordinary data, a message, a file, an image, but arranges it so the computer runs part of it as instructions. That is the essence of many serious attacks (buffer overflows, code injection), which you will study in later courses. It all starts here: instructions and data are made of the same stuff.

6. Why Security People Must Understand the Machine

You might wonder why a security course spends time on CPUs and memory. Because **you cannot secure, attack, or investigate what you do not understand.** Every area of security rests on the machine:

Security work	The machine knowledge it needs
Reverse engineering (course 17)	Reading a program's instructions and memory
Binary exploitation (course 18)	Bending how a program uses memory, especially the stack
Malware analysis (course 29)	Understanding executables, processes, and the OS
Digital forensics (course 28)	Recovering data from memory, disks, and deleted files
Defense and detection	Knowing what "normal" processes, files, and boot look like

Notice how often the later courses appear. This course is the trunk; those are the branches. When a forensic analyst recovers a deleted file, they are using what lesson 7 teaches about how files are stored. When an exploit developer overflows a buffer, they are using what lessons 6 and 11 teach about the stack. Skip the trunk, and the branches never hold.

There is also a simpler reason: understanding the machine removes fear. A computer is not magic. It is a fast, literal follower of numeric instructions. Once you see that clearly, security stops being mysterious and becomes something you can reason about.

7. A Map of This Course

Here is where we are going, and why the order makes sense.

Part	Lessons	What you will understand
Numbers and text	2–3	How numbers (binary, hex) and text (ASCII, Unicode, Base64) are stored
The machine	4–7	The CPU, memory, and storage; the operating system; processes; files
Programs and boot	8–10	How programs are built and run; how a computer starts; virtual machines
Memory, up close	11–12	A first look at C and memory, then a hands-on project on a real system

Each lesson builds on the one before. By the end, you will be able to open up a running computer, inspect its processes, read a file in raw form, follow it from power-on to desktop, and understand, in outline, how a program lives in memory. That is the foundation every later security course assumes.

One last framing, carried from Security Fundamentals: everything here is for understanding and defending. We look inside our *own* machines. Nothing in this course requires, or encourages, touching a computer that is not yours.

In the next lesson, we start at the very bottom, with the ons and offs themselves: bits, binary, and hexadecimal.

Common Mistakes

1. Thinking a computer "understands" what it does. It does not. It follows numeric instructions, literally and fast. That literalness is exactly what attackers exploit.

2. Believing software is separate from numbers. A program is numbers in memory, just like data. This is the root of many attacks.

3. Treating hardware and software as the same thing. Hardware is the machine; software is the instructions it runs. Attacks target different layers, so the distinction matters.

4. Assuming "it's just data, it's harmless." Because instructions and data share the same memory, carefully crafted data can be run as instructions. Input is never automatically safe.

5. Skipping the fundamentals to get to "the hacking." Reversing, exploitation, and forensics all rest on these basics. Without them, the advanced courses are memorized recipes, not understanding.

Summary

Idea	Remember
What a computer is	A machine that follows numeric instructions, step by step, very fast
The four jobs	Input, processing, output, storage
Hardware vs software	The physical machine vs the instructions it runs
Everything is numbers	Text, images, sound, and programs are all numbers, stored as on/off switches
Instructions and data	Both are numbers in the same memory (stored-program); this enables both flexibility and attacks
Why it matters	You cannot attack, defend, or investigate what you do not understand

Check Your Understanding

1. In one sentence, what is a computer, at its core?
2. Name the four things every computer does, with an example of each.
3. What is the difference between hardware and software?
4. "Inside a computer, everything is numbers." Give three different things that are stored as numbers.
5. What does the stored-program idea mean, and why does it make computers flexible?
6. How does the stored-program idea also enable a class of attacks?
7. Give two examples of security work and the machine knowledge each needs.

Answers

1. A computer is a machine that follows a list of instructions, step by step, very fast, to work with information.
2. Input (information comes in, e.g. typing), processing (working on it, e.g. calculating), output (a result goes out, e.g. the screen), storage (keeping it, e.g. saving a file).
3. Hardware is the physical machine (processor, memory, disk); software is the instructions (programs) that tell the hardware what to do.

4. Any three: text/letters, colors in a photo, sound, program instructions, images, video, documents.
5. A computer holds its instructions and data together in the same memory. It makes computers flexible because you give the machine new behavior just by giving it a new program, not by rebuilding it.
6. Because instructions and data are the same kind of thing in the same memory, a computer can be tricked into treating crafted data as instructions (e.g. buffer overflows, code injection).
7. Any two: reverse engineering needs reading a program's instructions/memory; forensics needs recovering data from memory and disks; exploitation needs understanding how programs use memory; malware analysis needs understanding executables, processes, and the OS.

Practice

No computer needed, just think and write.

1. Pick a everyday action on your phone (sending a photo, playing a song, paying with a QR code). Break it into the **four stages**: what is the input, the processing, the output, and the storage?
2. For that same action, name one thing that is stored as **numbers** even though it does not look like a number (for example, the photo's colors).
3. For that same action, name one of the four stages an **attacker** might target, and what they might gain.

A Worked Example

Paying with a QR code:

1. Input: the camera reads the QR code, and you enter your PIN. Processing: the app checks the code, talks to the bank, and confirms you have the money. Output: a "payment successful" screen and a receipt. Storage: the transaction is saved in your history and the bank's records.
2. The QR code itself is just numbers (the merchant's ID and the amount, encoded as a pattern of black and white squares, which are 1s and 0s).
3. An attacker might target the **input** by showing you a fake QR code that pays *them* instead of the shop, gaining your money. (Defending against exactly this is part of the Fraud and Banking Security course.)

Course mark icon from Material Symbols by Google, Apache License 2.0.